

2. **NLP Extraction** — Groq LLM pulls out the slot value and checks for injection
3. **Slot State Management** — tracks which slots are filled and which are pending
4. **RAG Retrieval** — ChromaDB is queried for the most relevant policy chunks
5. **Decision Engine** — LLM reasons over all available context and gives a verdict
6. **Persistence** — the final request is saved to SQLite and a confirmation is sent

2.2 Telegram Bot Layer

The bot is built with `python-telegram-bot v20`. It exposes three commands: `/start` to greet the user, `/request` to begin a new asset request conversation, and `/upload_rulebook` which is restricted to admins and allows uploading a new company policy PDF. Every incoming message is routed through the suspicion scorer first before reaching any other part of the system.

2.3 NLP Front-End

Each user reply is sent to the Groq API using the `llama-3.3-70b-versatile` model. A structured system prompt instructs the model to extract the target slot value, fix any typos or paraphrases in the input, check whether the message looks like a prompt injection attempt, and return a confidence score between 0 and 1.

If the confidence score is below 0.7, the bot re-asks the question with a helpful example. If the injection risk is flagged as high, the conversation is immediately frozen and the event is recorded in the audit log.

2.4 Slot Filling Engine

The bot collects five pieces of information from the employee: asset type, employee name, role, grade, and justification. These are called slots. The conversation handler moves through the slots one at a time, only advancing when a slot is filled with sufficient confidence.

Sessions automatically expire after ten minutes of inactivity. The employee can type `/cancel` at any point to reset the conversation. If the employee sends an image or a sticker mid-conversation, the bot sends a friendly redirect instead of crashing.

2.5 RAG Pipeline

When an admin uploads the company rulebook PDF, the following steps happen automatically. `PyMuPDF` extracts all the text from the document. `RecursiveCharacterTextSplitter`

breaks it into 400-token chunks with a 60-token overlap. `sentence-transformers` converts each chunk into a vector embedding. `ChromaDB` stores all embeddings locally on disk.

When a policy check is needed, the bot embeds the query and retrieves the top three most relevant chunks from ChromaDB. These chunks are injected directly into the decision engine prompt so that every approval or rejection cites a real passage from the company rulebook.

2.6 Decision Engine

The decision engine is the brain of MUGEN AI. It receives four inputs at once: the employee record from the mock HRIS, the five validated slot values, the top three policy chunks from RAG, and the product catalog showing current stock and pricing.

The Groq LLM reasons through all of this step by step and returns a structured JSON verdict. The verdict includes whether the request is approved, flagged, or rejected; a one-sentence reason that cites the relevant policy; a suggested alternative if the requested product is out of stock or over budget; and the exact rulebook passage that applies.

2.7 Suspicion Detection

This layer runs before all business logic and scores every message independently. It accumulates a risk score per session based on four signals.

- **Prompt injection patterns** add 80 points. Phrases like “ignore all instructions” or “you are now” trigger this.
- **After-hours activity** outside 07:00 to 20:00 adds 15 points.
- **Grade-model mismatch** adds 40 points. For example, an L1 employee requesting a high-tier workstation.
- **High velocity** adds 50 points. More than three requests in 24 hours triggers this.

A score above 80 blocks the session entirely. A score above 40 flags the request and sends an alert to the admin. Below 40, the conversation proceeds normally.

2.8 Persistence Layer

All data is saved to a SQLite database running in WAL mode using `aiosqlite` for async access. Two tables are maintained. The `asset_requests` table stores the complete validated request record. The `audit_events` table stores every suspicion trigger with the user ID,

timestamp, risk score, flags raised, and the raw message truncated to 200 characters for privacy.

3 Data Flow

Figure 2 shows the complete message sequence from the moment the employee sends a message to the moment a confirmation is returned. The flow below describes a successful approved request.

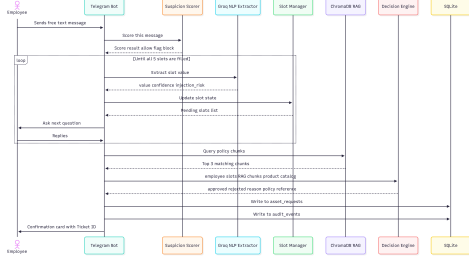


Figure 2: Message sequence from user input to SQLite persistence

The employee sends a free-text message to the Telegram bot. The suspicion scorer evaluates it and, if the score is below the block threshold, passes it forward. The Groq NLP extractor identifies the slot value, normalises the input, and returns a confidence score. If confidence is acceptable, the slot is marked as filled.

This cycle repeats until all five slots are filled. At that point, the RAG retriever embeds the policy query and fetches the three most relevant policy chunks from ChromaDB. The decision engine receives the employee record, the filled slots, the policy chunks, and the product catalog. It returns a structured verdict.

If the request is approved, the full record is written to the `asset_requests` table and a formatted confirmation card is sent to the employee on Telegram. All suspicion events from the entire conversation are written to `audit_events` in real time throughout the flow.

4 Technology Stack

Table 1 lists every technology used in MUGEN AI and the role it plays in the system.

Table 1: MUGEN AI Technology Stack

Layer	Technology
Runtime	Python 3.11
Bot Framework	python-telegram-bot v20
LLM Provider	Groq API, llama-3.3-70b-versatile
Chain Management	LangChain LCEL
PDF Extraction	PyMuPDF (fitz)
Text Chunking	RecursiveCharacterTextSplitter
Embeddings	sentence-transformers all-MiniLM-L6-v2
Vector Store	ChromaDB (local, persistent)
Database	SQLite with WAL mode,aiosqlite
Data Validation	Pydantic v2
Configuration	python-dotenv
Source Control	GitHub